

# PROJET DOOM-LIKE P2P

COMPTE-RENDU D'AVANCEMENT : ITÉRATION 5

Équipe : Chailan Cyprian, Gros Geoffrey, Lounici Ilyes,  
Ray Marcelin  
3 avril 2026

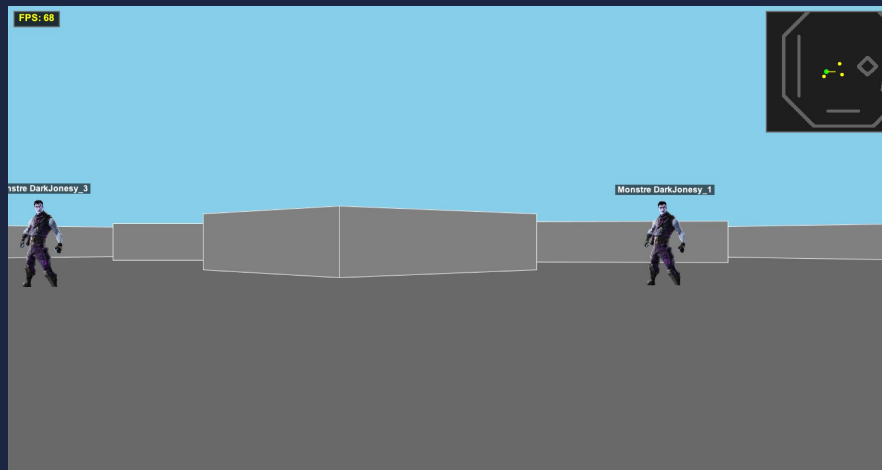
# Sommaire

- **Rappel du projet et objectifs de l'itération**
- **Fonctionnalités actuelles**
  - **Amélioration documentation**
  - **MiniMap**
  - **Synchronisation des monstres en Réseau**
  - **BSP - complications rendu des textures/annulé**

# Objectif de l'itération 5

But principal : Avoir un livrable

- Avoir des monstres en réseau sans lag et un comportement de groupe
- Avoir une application qui met en valeur tout ce que nous avons développé



# README - Makefile

Guide d'utilisation dans le Readme. Le Makefile permet en une seule commande, pouvoir compiler le projet et démarrer l'application.

## ProjetTutoreDOOM

### Informations Générales

- Nom du projet : ProjetTutoreDOOM
- Auteurs : Cyprian, Ilyès, Marcelin, Geoffrey

### Installation

Le projet est une application Java standard.

### Prérequis

- Java JDK 25 ou supérieur.
- GNU Make (optionnel, pour utiliser le Makefile).
- Un IDE (IntelliJ IDEA, Eclipse, VS Code) ou un terminal.

### Procédure

### ProjetTutoreDOOM - Cibles disponibles

=====

|                                |                                          |
|--------------------------------|------------------------------------------|
| <code>make</code>              | Compile toutes les sources               |
| <code>make run</code>          | Compile et lance le jeu multijoueur      |
| <code>make run-rrt</code>      | Compile et lance la demo RRT             |
| <code>make run-rrt-prog</code> | Compile et lance la demo RRT progressive |
| <code>make clean</code>        | Supprime les fichiers compilés           |
| <code>make help</code>         | Affiche cette aide                       |

*Makefile help*

*Readme.md*

# Minimap

## L'objectif:

- Réaliser une MiniMap qui permet de voir les monstres / joueurs
- Facilement se rendre compte du comportement des monstres et de l'environnement 2D
- Plus facilement debuggable



*Exemple minimap*

# Minimap

Bug découvert grâce à la MiniMap:

- Bug de collision, les monstres traversaient les murs dans certains cas.
- Logique de collision Raycasting/BSP oublié pendant un merge.



# Fonctionnement Automate

L'automate dicte *quoi* faire (l'intention), mais le *comment* (le mouvement réel) est géré par d'autres classes de notre architecture :

- **Steering Behaviors** : Pour le calcul de l'orientation fluide et la physique de déplacement (vitesse, direction)
- **Algorithme RRT (Rapidly-exploring Random Tree)** : Permet au monstre de calculer une route complexe pour contourner les murs (Pathfinding) lors de la poursuite ou de la patrouille

# L'Automate à états finis - IA des Monstres

Nous avons implémenté une “Intelligence Artificielle” basée sur un automate d'états finis pour gérer les prises de décision du monstre.

La boucle de jeu de l'IA évalue en permanence les conditions pour basculer entre :

- 1) **Attente** : Le monstre patiente un certain temps.
- 2) **Patrouille** : Déplacement régulier entre un point A et un point B.
- 3) **Poursuite** : S'active instantanément si une cible est détectée. Le monstre traque le joueur jusqu'à ce que la cible soit perdue ou capturée.
- 4) **Alerte** : Un monstre alerte les autres monstres qui se déplacent vers la position du signalement.

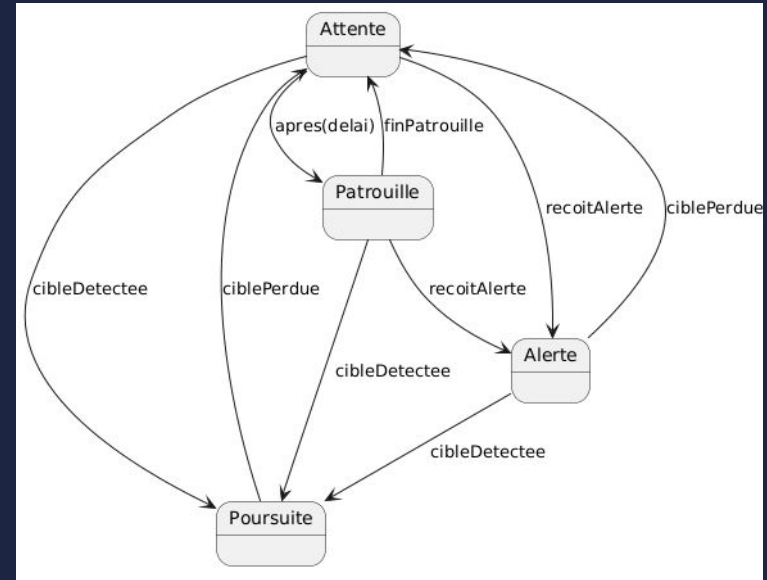
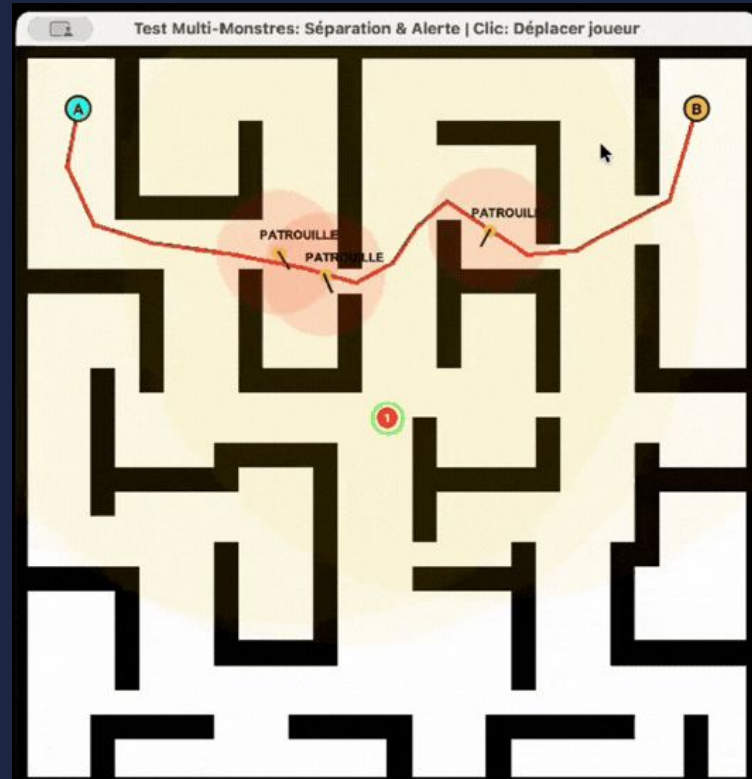


Diagramme dispo dans notre documentation  
*Automate.puml*



# Fonctionnement Automate



## BSP - complication des textures

Les murs BSP posent de nombreux défis techniques pour le rendu des textures, car plusieurs paramètres interdépendants doivent être gérés simultanément.

- **Découpage BSP & Raccords** : calculer l'offset pour chaque segment pour avoir une texture qui se continue naturellement d'un mur à l'autre.
- **Alignement** : Il faut que la texture soit plaqué sur le mur et pas un peu décalé, ou une texture qui déborde du mur. Ce qui peut créer des artefacts visuelle.
- **Angles** : les angles entre les murs peuvent créer des textures distordu.
- **Stabilité au mouvement** : Au déplacement du joueurs, on veut éviter que la texture slide ou saute au lieu de rester fixe sur le mur.

# Fonctionnement du monstre en réseau

- Utilisation d'une séparation sur les monstres dans le but de rendre le groupe plus naturel
- Utilisation d'un thread pour la mise à jour des monstres
- Ajout de slot autour du joueur afin que les monstres attaquent le joueur de plusieurs directions différentes
- Un joueur s'occupe de mettre à jour tous les monstres et d'envoyer leur position aux autres

# Planning pour l'itération finale

- Trouver et réparer le plus de bugs possible
- Tester sur les machines de l'IUT
- Préparer des maps de tests pour présenter aux étudiants la semaine prochaine
- Finir la doc final

# Questions ?



Merci pour votre attention